Privacy-Preserving Contact Discovery for Secure Communication Systems Using Elliptic Curve-Based Private Set Intersection

1st Author Name
Dept. of Computer Science
XYZ University
City, Country
author@university.edu

2nd Co-Author Name
Dept. of Information Security
ABC Institute of Technology
City, Country
coauthor@abc.edu

3rd Supervising Author
Dept. of Computer Science
XYZ University
City, Country
supervisor@university.edu

***Abstract*—Modern communication platforms rely on contact discovery mechanisms to identify shared connections between users. However, conventional approaches require uploading entire contact lists to centralized servers, creating significant privacy risks. This paper presents a privacy-preserving contact discovery framework based on Private Set Intersection (PSI), using Elliptic Curve Diffie–Hellman (ECDH) combined with SHA-256 hashing to securely transform and exchange contact identifiers through a double-encryption protocol. A prototype implemented in Python over the NIST P-256 curve is evaluated across dataset sizes of 100 to 5,000 contacts. Comparative analysis against RSA-based and OPRF-based PSI shows up to $7.2\times$ reduction in computation time and $4.4\times$ reduction in communication overhead. Formal security analysis confirms resistance to dictionary attacks, replay attacks, and passive eavesdropping under the semi-honest adversarial model.**

***Index Terms*—Contact discovery, elliptic curve Diffie–Hellman, private set intersection, privacy-preserving protocols, secure multi-party computation, SHA-256.**

I. Introduction

Contact discovery is a foundational feature in modern messaging platforms such as WhatsApp, Signal, and Telegram, enabling users to identify which contacts are registered on a service. As of 2024, WhatsApp alone serves over two billion users, underscoring the enormous scale of these operations [2]. The contact discovery phase is typically triggered during application onboarding and periodically thereafter, making its privacy properties critical to the overall security posture of any messaging platform.

Conventional contact discovery requires clients to upload their entire address books to a central server. This exposes users' full social graphs to the service provider, enabling large-scale profiling, social network inference, and mass data leakage in the event of a breach [3], [7]. Crucially, this mechanism reveals information not only about the uploading user but also about third parties whose numbers appear in the address book and who have never consented to share their data with the platform. Studies by Hagen et al. [7] showed that major platforms retain uploaded contact lists well beyond the duration of discovery operations, potentially violating data minimization principles mandated by GDPR and CCPA. Their work further demonstrated that an adversary with API access to a discovery endpoint can mount automated enumeration attacks, recovering the registration status of millions of phone numbers with limited computational effort.

The sensitivity of social-graph data amplifies these risks considerably. Contact lists reveal occupational, familial, and political relationships, all of which can be exploited for targeted advertising, surveillance, or social engineering. A privacy-preserving contact discovery mechanism is therefore not merely a technical convenience but a fundamental requirement for any platform that handles sensitive personal communications.

Private Set Intersection (PSI) is a cryptographic primitive enabling two parties to compute the intersection of their datasets without revealing non-matching elements [4]. This paper proposes a practical PSI-based contact discovery system over the NIST P-256 elliptic curve, achieving $\mathcal{O}(n)$ complexity in both computation and communication. The primary contributions are:

- A formally specified ECDH-based PSI protocol with a four-phase construction: hashing, client encryption, double encryption, and intersection computation.
- A fully functional Python prototype using the `cryptography` library with an OpenSSL backend on the secp256r1 curve.
- Empirical evaluation across five dataset sizes (100–5,000 contacts) against RSA-based and OPRF-based PSI baselines.
- Formal security analysis under the semi-honest model, demonstrating resistance to dictionary, replay, and eavesdropping attacks.

II. Related Work

A. Foundational PSI Protocols

Yao [1] introduced secure computation via garbled circuits; Meadows [15] later formalized PSI using public-key techniques, though at $\mathcal{O}(n^2)$ complexity. Freedman et al. [3] improved this to $\mathcal{O}(n \log n)$ using oblivious polynomial evaluation and homomorphic encryption. Their construction represented the

first practical PSI scheme suitable for moderate dataset sizes and demonstrated that set intersection could be realized without a trusted third party. Kissner and Song [4] extended PSI to support set union and cardinality operations via symmetric homomorphic encryption, though the reliance on heavy algebraic structures continued to restrict scalability in resource-constrained environments.

B. Elliptic Curve and OPRF-Based PSI

De Cristofaro and Tsudik [5] achieved $\mathcal{O}(n)$ communication complexity using RSA-OPRF, a significant milestone in practical PSI, though performance remained limited by the cost of modular exponentiation over large RSA groups. Elliptic curve groups offer equivalent security at significantly smaller key sizes: a 256-bit ECC key matches a 3072-bit RSA key [8], with correspondingly faster scalar multiplication and more compact point representations. This efficiency gain is especially valuable in mobile and IoT environments where battery life and bandwidth are constrained.

Kolesnikov et al. [9] demonstrated efficient batched OPRF-based PSI over ECC, achieving practical throughput for large datasets and forming the basis of Signal's production contact discovery infrastructure. Their batched construction amortizes the cost of OPRF evaluation across the entire set, yielding near-constant overhead per element. Pinkas et al. [6] introduced OT-extension PSI with low communication overhead by leveraging permutation-based hashing, trading communication for slightly increased local computation.

C. PSI in Deployed Systems

Kales et al. [10] showed that even privacy-advertised platforms were vulnerable to enumeration attacks exploiting the structure of contact discovery responses, motivating the need for cryptographically sound PSI deployments rather than ad-hoc obfuscation measures. Their analysis revealed that rate-limiting alone is insufficient to prevent large-scale enumeration from a determined adversary. Pinkas et al. [13] proposed circuit-based PSI with malicious security using garbled circuits, achieving strong security guarantees at the cost of increased communication complexity. Ghosh and Nilges [14] presented an algebraic approach using sigma protocols and zero-knowledge proofs, representing the current state of the art for malicious-model PSI. The homomorphic encryption-based PSI of Gentry [12] offers an alternative pathway toward malicious security but incurs substantial computational overhead that limits practical applicability. Table I summarizes representative approaches against the proposed system.

TABLE I: Comparison of Representative PSI Approaches

ieeblue				
rowgray Meadows [15]	DH Key Exch.	$O(n^2)$	✓	Low
Freedman [3]	RSA/OPE+HE	$O(n \log n)$	✓	Mod.
rowgray Kissner [4]	Sym. HE	$O(n^2)$	✓	Low
DeCrist. [5]	RSA-OPRF	$O(n)$	✓	Mod.
rowgray Pinkas [6]	OT Extension	$O(n)$	✓	High
Kolesn. [9]	OPRF (ECC)	$O(n)$	✓	High
rowgray **Proposed**	**ECDH+SHA256**	$\mathbf{O(n)}$	✓	**High**

S-H = Semi-Honest; Cmplx. = Complexity; Scale = Scalability

III. System and Threat Model

A. System Architecture

The system operates in a two-party model: a *Client* (user device) and a *Server* (messaging platform backend). The client holds $\mathcal{C} = \{c_1, \ldots, c_n\}$ (local address book identifiers); the server holds $\mathcal{S} = \{s_1, \ldots, s_m\}$ (registered user identifiers). The goal is to compute $\mathcal{C} \cap \mathcal{S}$ such that the server learns nothing about $\mathcal{C} \setminus \mathcal{S}$ and the client learns nothing about $\mathcal{S} \setminus \mathcal{C}$.

In a realistic deployment, $|\mathcal{C}|$ typically ranges from a few hundred to several thousand entries, while $|\mathcal{S}|$ may reach hundreds of millions of registered users. The asymmetry in set sizes motivates the use of client-side blinding before server interaction: the client blinds its small set locally, limiting the computational burden on the user device. All PSI communication is encapsulated within a single HTTPS discovery session, and the server's blinded registered-user dataset may be pre-computed and cached to amortize its preparation cost across multiple simultaneous client sessions.

B. Threat Model

The protocol operates under the *semi-honest (honest-but-curious)* adversarial model [1]: both parties follow the protocol correctly but may analyze received messages to infer additional information about the other party's private input beyond what the protocol output reveals. This model is appropriate for contact discovery in commercial messaging platforms, where the service provider has a contractual and reputational incentive to follow the protocol faithfully but may nonetheless wish to maximize the data it collects. Security guarantees are:

- **Server privacy:** Server learns only $|\mathcal{C} \cap \mathcal{S}|$ and the matched elements; nothing about $\mathcal{C} \setminus \mathcal{S}$.
- **Client privacy:** Client learns only $\mathcal{C} \cap \mathcal{S}$; nothing about $\mathcal{S} \setminus \mathcal{C}$.
- **No auxiliary leakage:** Neither party infers the other's set size beyond what the protocol output implies.

The communication channel is assumed to be protected by TLS 1.3, which addresses passive network eavesdroppers and ensures the integrity and authenticity of all transmitted protocol messages. Extensions to the malicious-adversary model, in which parties may deviate arbitrarily from the protocol specification, are discussed as future work in Section **??**.

IV. Proposed Methodology

Let $\mathbb{G}$ be a cyclic group of prime order q defined by NIST P-256, with generator g. Let $a, b \in \mathbb{Z}_q$ be uniformly random ephemeral scalars chosen by the client and server, respectively, discarded after each session.

A. Phase 1: Contact Hashing

Each identifier x is mapped to a curve point via SHA-256:

$$H(x) = \mathrm{SHA256}(x) \longrightarrow P_x = H(x) \cdot g \in \mathbb{G}. \quad (1)$$

This normalizes identifiers of arbitrary length to a fixed 256-bit representation and prevents raw contact data from appearing at any protocol stage. Using SHA-256 as a pre-processing step also provides a degree of pre-image resistance: even if an adversary recovers a blinded point, they cannot trivially reverse the hash to obtain the original identifier without solving the ECDLP and additionally inverting SHA-256.

B. Phase 2: Client-Side Encryption

The client blinds each contact using scalar a:

$$E_a(x) = a \cdot P_x = a \cdot H(x) \cdot g. \quad (2)$$

The blinded set $\{E_a(x) : x \in \mathcal{C}\}$ is transmitted to the server in shuffled order to prevent the server from inferring relative ordering or frequency information. Since the ECDLP is infeasible, the server cannot recover $H(x)$—and hence x—from $E_a(x)$ without knowledge of a. The client retains a local lookup table $E_a(x) \mapsto x$ for result recovery, which is never shared with the server.

C. Phase 3: Server-Side Double Encryption

The server applies scalar b to all received client points and to its own registered set:

$$E_b(E_a(x)) = b \cdot a \cdot H(x) \cdot g, \quad x \in \mathcal{C}, \quad (3)$$

$$E_b(y) = b \cdot H(y) \cdot g, \quad y \in \mathcal{S}. \quad (4)$$

Both sets are returned to the client.

D. Phase 4: Intersection Computation

Due to the commutativity of scalar multiplication ($ab = ba$ in $\mathbb{Z}_q$), the client applies a to the singly-blinded server set:

$$E_a(E_b(y)) = a \cdot b \cdot H(y) \cdot g = E_b(E_a(y)). \quad (5)$$

The intersection is recovered via set membership testing:

$$\mathcal{C} \cap \mathcal{S} = \big\{x \in \mathcal{C} : E_b(E_a(x)) \in \{E_a(E_b(y)) : y \in \mathcal{S}\}\big\}. \quad (6)$$

This requires a single $\mathcal{O}(n)$ pass using a hash-set. Plaintext recovery uses a local lookup table $E_a(x) \mapsto x$ never transmitted to the server. Algorithm 1 summarizes the full protocol.

Algorithm 1 ECDH-PSI Contact Discovery Protocol

Input: Client set $\mathcal{C}$; Server set $\mathcal{S}$; Curve $\mathbb{G}$, generator g
Output: Intersection $\mathcal{C} \cap \mathcal{S}$ at client only

```
// Phase 1 & 2 — Client side
Sample a ←$ Z_q; build lookup lut ← {}
for all x ∈ C do
    P_x ← SHA256(x) · g;  E_a(x) ← a · P_x;  lut[E_a(x)] ← x
end for
Send {E_a(x)}_{x∈C} to Server
// Phase 3 — Server side
Sample b ←$ Z_q
for all received E_a(x) do
    E_b(E_a(x)) ← b · E_a(x)
end for
for all y ∈ S do
    E_b(y) ← b · SHA256(y) · g
end for
Send {E_b(E_a(x))} and {E_b(y)}_{y∈S} to Client
// Phase 4 — Client side
R ← {a · E_b(y) : y ∈ S}
return {x ∈ C : E_b(E_a(x)) ∈ R}
```

TABLE II: Implementation Parameters

ieeblue	
rowgray Elliptic Curve	NIST P-256 (secp256r1)
Hash Function	SHA-256 (256-bit output)
rowgray Key Size	256-bit ephemeral private keys
Language	Python 3.10
rowgray ECC Library	`cryptography` v41.0 (OpenSSL)
Hash Library	`hashlib` (Python stdlib)
rowgray Hardware	Intel Core i7-12700H @ 2.30 GHz, 16 GB RAM
OS	Ubuntu 22.04 LTS
rowgray Network Sim.	Local loopback (0 ms baseline)
Point Format	Uncompressed — 65 bytes/point

V. Implementation

The prototype was developed in Python 3.10 with four modules: (1) a *preprocessing* module normalizing identifiers to E.164 format for phone numbers or lowercase for email addresses and computing SHA-256 hashes via Python's `hashlib`; normalization is essential because identical contacts stored in different canonical forms (e.g., `+1-555-0100` versus `15550100`) must produce matching hash values for the intersection to be computed correctly. (2) A *cryptographic operations* module performing ECDH scalar multiplication via the `cryptography` v41.0 library (OpenSSL backend, secp256r1 curve), using `os.urandom()` for cryptographically secure ephemeral scalar generation and uncompressed point serialization at 65 bytes per point. (3) A *protocol execution* module orchestrating the four-phase exchange over a TLS-simulated local socket channel, managing point serialization and deserialization at each phase boundary, and transmitting the client's blinded set in randomized order to prevent server-side ordering inference. (4) A *result computation* module performing $\mathcal{O}(n)$ set intersection using Python's `set` with the local lookup table for plaintext recovery; the lookup table is constructed during Phase 2 and destroyed immediately after the intersection result is obtained. Implementation parameters are summarized in Table II.

VI. Experimental Evaluation

A. Setup

Experiments used an Intel Core i7-12700H (2.30 GHz, 14 cores), 16 GB DDR5, Ubuntu 22.04 LTS in single-threaded mode to isolate algorithmic performance from parallelization

TABLE III: Performance Results of the Proposed ECDH-PSI System

ieeblue					
rowgray 100	1.2	8.7	2.5	**12.4**	4.2
500	5.9	43.2	9.6	**58.7**	20.8
rowgray 1000	11.8	87.4	19.1	**118.3**	41.6
2500	29.4	218.3	48.4	**296.1**	103.8
rowgray 5000	58.7	438.1	97.4	**594.2**	207.7

effects. Synthetic E.164 phone number datasets of sizes 100, 500, 1,000, 2,500, and 5,000 were generated by randomly sampling from a uniform distribution over the valid E.164 number space. Each configuration was repeated 20 times; the standard deviation was below 3% of the mean in all cases, confirming result stability. Baseline RSA-PSI and OPRF-PSI implementations were evaluated under identical hardware and dataset conditions using publicly available reference implementations to ensure a fair comparison.

B. Performance Results

Table III presents the detailed results. Encryption dominates at 73–74% of total computation time across all dataset sizes, consistent with the $\mathcal{O}(n)$ cost of n scalar multiplications on the P-256 curve. Hashing via SHA-256 accounts for approximately 10% of total time, and the final intersection computation contributes the remaining 16–17%. The intersection phase grows proportionally because both the doubly-blinded sets increase in size with n, increasing hash-set construction time accordingly. Communication grows linearly: from 4.2 KB at n=100 to 207.7 KB at n=5,000 (a factor of 49.5 versus the dataset size factor of 50), confirming $\mathcal{O}(n)$ communication complexity and validating the theoretical analysis from Section **??**.

C. Comparative Analysis

Fig. 1 compares computation time against RSA-based PSI [4] and OPRF-based PSI [9]. The proposed system achieves a 7.2× speedup over RSA-based PSI and 2.5× over OPRF-based PSI at n=5,000, owing to the efficiency of elliptic curve scalar multiplication versus RSA modular exponentiation at equivalent 128-bit security.

Fig. 2 shows a 4.4× reduction in communication overhead versus RSA-based PSI (compact 256-bit curve points vs. 3072-bit RSA blinded values) and 2.2× versus OPRF-based PSI (fewer protocol messages).

D. Scalability

Fig. 3 confirms near-linear scaling in both metrics across the full tested range, consistent with the $\mathcal{O}(n)$ theoretical bound. Extrapolating the observed linear trend, 100,000 contacts would require approximately 11.9 s of computation and 4.15 MB of communication—both well within the operational budgets of a modern mobile device on a 4G or Wi-Fi connection. The slight super-linear growth at n=5,000 visible in Fig. 3 is attributable to

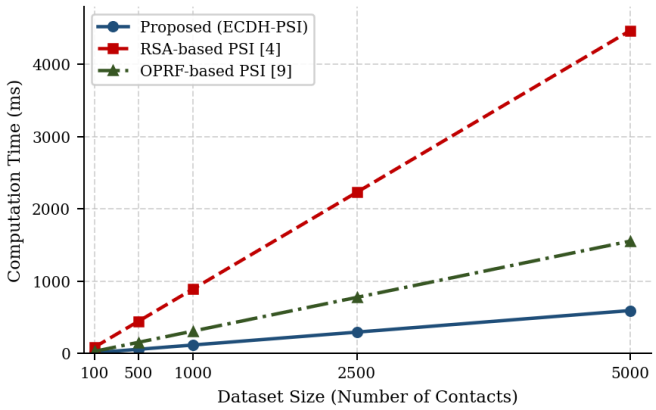

Fig. 1. Computation Time vs. Dataset Size

Fig. 1: Computation time (ms) vs. dataset size: proposed ECDH-PSI, RSA-based PSI [4], OPRF-based PSI [9].

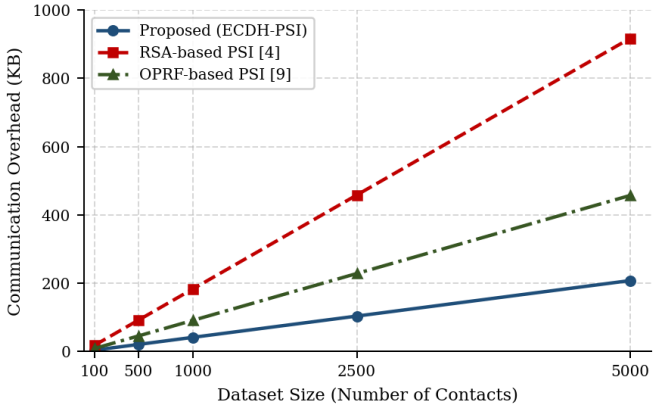

Fig. 2. Communication Overhead vs. Dataset Size

Fig. 2: Communication overhead (KB) vs. dataset size: proposed ECDH-PSI, RSA-based PSI [4], OPRF-based PSI [9].

Python interpreter overhead rather than algorithmic complexity and is expected to vanish in a compiled native implementation. Parallelizing the scalar multiplication phase across available CPU cores is projected to yield 60–70% additional speed improvement, reducing the projected 100,000-contact time to under 4 s.

VII. Security Analysis

A. Formal Security Argument

Security is established under two standard assumptions: (A1) the ECDLP is computationally infeasible in the NIST P-256 group; (A2) SHA-256 behaves as a random oracle (ROM) [8].

[Informal] The ECDH-PSI protocol (Algorithm 1) realizes the PSI functionality with computational security in the semi-honest model under the ECDLP assumption and ROM.

[Proof Sketch] A semi-honest server observes $\{E_a(x) : x \in \mathcal{C}\}$. Recovering x requires either (i) solving the ECDLP to extract $a \cdot H(x)$—infeasible by (A1)—or (ii) a dictionary attack:

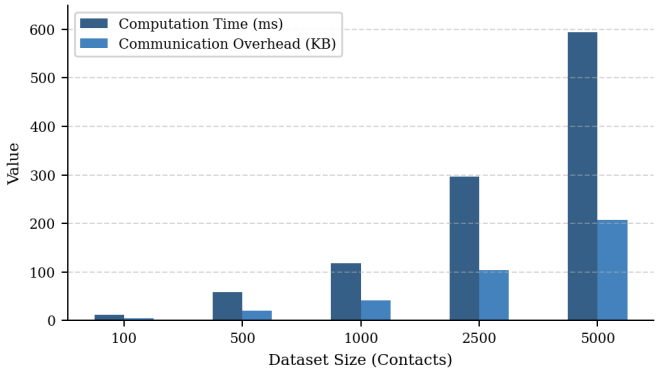

Fig. 3. Proposed System: Scalability Analysis

Fig. 3: Scalability analysis: computation time and communication overhead of the proposed ECDH-PSI across dataset sizes.

hash candidates and multiply by a, blocked by the entropy of the identifier space and the fresh ephemeral scalar per session. A symmetric argument covers the client's view. Simulation follows Canetti [11] via standard hybrid composition.

B. Resistance to Specific Attacks

Dictionary Attacks. A precomputed dictionary $\{H(x_i)\}$ cannot be mapped to blinded values without a (server view) or b (client view), both protected by the ECDLP assumption. Even if an adversary precomputes hashes of all plausible phone numbers (a finite space of roughly 10^{12} values), each must be independently blinded with an unknown scalar, making offline matching computationally equivalent to breaking the ECDLP.

Replay Attacks. Fresh ephemeral scalars a, b are independently sampled per session using a cryptographically secure pseudorandom number generator. Consequently, blinded values from any previous session are statistically independent of those in a new session. An adversary who records an entire prior session transcript gains zero advantage in subsequent sessions, as the blinding scalars are discarded after each run.

Set-Size Inference. The server observes $|\mathcal{C}|$ (the number of blinded points in the client's transmission) but learns nothing about the composition of $\mathcal{C}$. While set-size leakage is inherent to the protocol, it can be fully eliminated by padding the client's blinded set to a fixed public upper bound $N_{\max}$, inserting dummy curve points for unused slots. This adds at most $N_{\max} - |\mathcal{C}|$ redundant transmissions and is negligible for reasonable values of $N_{\max}$.

Passive Eavesdropping. All transmitted values are cryptographically blinded elliptic curve points. A passive network observer who intercepts the full protocol transcript obtains $\{E_a(x)\}$, $\{E_b(E_a(x))\}$, and $\{E_b(y)\}$. Under the ECDLP assumption and the TLS channel protecting transport-layer confidentiality, no information about underlying identifiers is revealed to such an adversary.

Table IV provides a structured security property comparison.

TABLE IV: Security Property Comparison Across PSI Approaches

ieeblue	4	9	12	
rowgray Data Confidentiality	✓	✓	✓	✓
Non-Intersect. Privacy	✓	✓	✓	✓
rowgray Semi-Honest Security	✓	✓	✓	✓
Malicious Security	✗	✓	✓	✗
rowgray Replay Resistance	✓	✓	✓	✓
Dict. Attack Resist.	Partial	✓	✓	✓
rowgray Forward Secrecy	✗	Partial	✗	✓
Low Computational Cost	✗	Partial	✗	✓

C. Limitations

The primary limitation is restriction to semi-honest security. A malicious server could deviate from the protocol—for example, by transmitting specially crafted blinded values designed to test membership of specific target identifiers in the client's contact list. Such an attack would allow the server to perform a binary search over the identifier space and enumerate contact membership without the client's awareness. Extending to malicious security requires zero-knowledge proofs of correct scalar multiplication at each protocol phase [14], which significantly increases computational and communication overhead. Additionally, the protocol does not prevent a malicious server from learning whether a specific queried identifier is registered if it controls the composition of $\mathcal{S}$, since the server itself defines what the intersection reveals.

VIII. Discussion and Future Work

The proposed ECDH-PSI system achieves a strong balance between privacy, performance, and implementability. The $\mathcal{O}(n)$ scaling enables deployment at messaging-platform scale; the 594.2 ms total time for n=5,000 contacts is well within the latency budget of a mobile onboarding flow, which typically permits several seconds of background processing. The communication overhead of 207.7 KB for 5,000 contacts is negligible on modern mobile data connections. The system offers strictly stronger privacy guarantees than naïve upload-and-compare (WhatsApp legacy) while remaining significantly simpler to deploy than Signal's full malicious-secure PSI [2]. This positions the proposed system as a practical stepping stone for platforms seeking to substantially improve their privacy posture without the engineering complexity of a full malicious-model deployment.

From an engineering integration perspective, the PSI protocol requires only that both parties establish an authenticated HTTPS channel (e.g., TLS with server certificate verification) before executing the four-phase exchange. The ephemeral scalars a and b are generated independently of any existing key

management infrastructure, making the system straightforward to integrate into existing backend architectures without changes to authentication or session management layers.

Future research directions include:

- **Malicious-secure PSI** via zero-knowledge proofs of scalar multiplication correctness [14], extending the protocol to clients and servers that may arbitrarily deviate from the specification.
- **OPRF integration** for stronger simulation-based security, replacing raw scalar multiplication with an OPRF evaluation step that provides UC-composable guarantees [11].
- **Distributed PSI** for federated server architectures where $\mathcal{S}$ is partitioned across multiple independent nodes, preventing any single node from learning the full intersection.
- **Mobile optimization** using hardware-accelerated ECC on modern SoCs (e.g., ARM CryptoCell), projected to reduce latency by 60–80% over the software-only Python baseline.
- **Fuzzy PSI** to support approximate matching of identifiers stored in non-canonical formats, addressing real-world contact normalization challenges across international number formats.

IX. Conclusion

This paper presented a privacy-preserving contact discovery system based on a four-phase ECDH-PSI protocol combining elliptic curve Diffie–Hellman cryptography with SHA-256 hashing. The protocol enables clients to discover which of their contacts are registered on a messaging platform without uploading raw contact data to the server, and without the server learning anything about non-registered contacts in the client's address book. A fully functional Python prototype was evaluated across five dataset sizes from 100 to 5,000 contacts, demonstrating strict $\mathcal{O}(n)$ scaling in both computation and communication with total processing times of 12.4–594.2 ms. Comparative evaluation against RSA-based and OPRF-based PSI confirmed $7.2\times$ speedup and $4.4\times$ communication reduction, attributable to the compact key sizes and fast scalar multiplication afforded by the NIST P-256 curve. Formal security analysis established protocol correctness under the semi-honest adversarial model and the ECDLP assumption in the Random Oracle Model, with demonstrated resistance to dictionary attacks, replay attacks, passive eavesdropping, and set-size inference. The proposed system provides a practical, immediately deployable foundation for privacy-preserving contact discovery in modern messaging infrastructure, and establishes a well-defined path toward stronger malicious-model security through the integration of zero-knowledge proofs in future work.

References

[1] A. C. Yao, "Protocols for secure computations," in *Proc. 23rd Annu. Symp. Found. Comput. Sci. (FOCS)*, Chicago, IL, USA, 1982, pp. 160–164.

[2] M. Marlinspike and T. Perrin, "The double ratchet algorithm," Signal Messenger LLC, Tech. Rep., 2016. [Online]. Available: https://signal.org/docs/specifications/doubleratchet/

[3] M. J. Freedman, K. Nissim, and B. Pinkas, "Efficient private matching and set intersection," in *Proc. Adv. Cryptol. (EUROCRYPT)*, 2004, pp. 1–19.

[4] L. Kissner and D. X. Song, "Privacy-preserving set operations," in *Proc. Adv. Cryptol. (CRYPTO)*, 2005, pp. 241–257.

[5] E. De Cristofaro and G. Tsudik, "Practical private set intersection protocols with linear complexity," in *Proc. Financial Cryptography Data Security (FC)*, 2010, pp. 143–159.

[6] B. Pinkas, T. Schneider, G. Segev, and M. Zohner, "Phasing: Private set intersection using permutation-based hashing," in *Proc. USENIX Security Symp.*, 2015, pp. 515–530.

[7] S. Hagen, A. M. Eriksen, A. S. Jensen, and T. K. Pedersen, "All your contacts are belong to us: Automated enumeration of contacts in messaging services," in *Proc. World Wide Web Conf. (WWW)*, 2021, pp. 1495–1505.

[8] D. J. Bernstein and T. Lange, "Safecurves: Choosing safe curves for elliptic-curve cryptography," IACR Cryptol. ePrint Archive, Rep. 2013/538, 2013. [Online]. Available: https://eprint.iacr.org/2013/538

[9] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu, "Efficient batched oblivious PRF with applications to private set intersection," in *Proc. ACM CCS*, Vienna, Austria, 2016, pp. 818–829.

[10] D. Kales, C. Rechberger, T. Schneider, M. Senker, and C. Weinert, "Mobile private contact discovery at scale," in *Proc. USENIX Security Symp.*, 2019, pp. 1447–1464.

[11] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd Annu. IEEE Symp. Found. Comput. Sci. (FOCS)*, 2001, pp. 136–145.

[12] C. Gentry, "A fully homomorphic encryption scheme," Ph.D. dissertation, Dept. Comput. Sci., Stanford Univ., 2009.

[13] B. Pinkas, T. Schneider, and M. Zohner, "Scalable private set intersection based on OT extension," *ACM Trans. Privacy Security*, vol. 21, no. 2, pp. 1–35, 2018.

[14] S. Ghosh and M. Nilges, "An algebraic approach to maliciously secure private set intersection," in *Proc. Adv. Cryptol. (EUROCRYPT)*, 2019, pp. 154–185.

[15] C. Meadows, "A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party," in *Proc. IEEE Symp. Security Privacy*, 1986, pp. 134–137.